

Tuple and Dictionaries

Krishna Kumar Mohbey¹, Malika Acharya¹

¹ Computer Science, Central University of Rajasthan, India

Abstract

Another important data type of Python is dictionary. In the previous chapter, we acquainted the readers with the lists and strings. In this chapter, we shall discuss dictionaries and lists. lists are the ordered collection of objects but dictionaries are an unordered collection of objects. dictionary values are referred to using key-value pair instead of positional offset. Due to this, they have found great usage in search tables, records and aggregation. Another concept introduced in this chapter is tuples. These are immutable like strings and represent a stable collection of arbitrary items.

Keywords: Hash tables, Immutable and mutable, Mappings.

INTRODUCTION

One of Python's most significant data types is called the tuple, which holds several components as an object. It is also known as an immutable data type, which indicates that its members cannot be altered in any way after they have been set. A dictionary is another data type that may be used in Python, and it functions similarly to tuples and lists. An associative array is another name for this structure. A dictionary is a collection of “key-value pairs”. Each key-value pair connects the key to the value that corresponds to it. In this chapter, you will learn about various operations and attributes that can be used with dictionaries and tuples. In addition, the many features and actions of dictionaries and tuples are broken down and illustrated using examples and codes.

TUPLE

A tuple is a group of ordered, immutable items arranged in a specific way. Tuples are structured similarly to lists and strings. In this context, the definition of immutable is that the components of the tuple are not subject to change. Once a tuple has been generated, it is not possible to add or remove items from the tuple. Even we cannot change the order in which the tuple members are presented. In addition, the length of the tuple cannot be altered. It is necessary to generate a new tuple if we wish to modify an existing one by adding or taking something away from it. Unlike lists, tuples are denoted by parentheses and cannot be edited. Tuples are a sort of sequence equivalent to strings in terms of structure. Tuples

can hold any components, in contrast to strings, which can only store characters. It indicates that the tuple contains either a list of students' names or employee IDs, depending on which one is selected. Tuples may also be used to store varied elements, implying that a single tuple can contain components of several data formats, such as decimal formats, integers, and characters. A sequence of music files, picture files, and other data types can also be stored in tuples.

To create a tuple in Python, all the elements are enclosed in () parenthesis, separated by a comma. A tuple can store heterogeneous data elements. Below are examples of creating tuples.

Example:

```
1.  # Create an empty tuple
2.  Tuple1 = ()
3.  # tuple with 5 elements
4.  Tuple2 = (10,20,30,40,50)
5.  print(Tuple2)
6.  # tuple of character elements
7.  Tuple3 = ('x', 'y', 'z')
8.  print(Tuple3)
9.  # tuple of strings
10. Tuple4 = ("Python", "Programming", "Book")
11. print(Tuple4)
12. # tuple of heterogeneous data elements
13. Tuple5 = (100, 20.123, "Python Programming")
14. print(Tuple5)
15. # tuple of string and List
16. Tuple6 = ("Python Book", [10, 20, 25])
17. print(Tuple6)
```

Output:

```
(10, 20, 30, 40, 50)
('x', 'y', 'z')
('Python', 'Programming', 'Book')
(100, 20.123, 'Python Programming')
('Python Book', [10, 20, 25])
```

To create a tuple of a single element, it should be followed by a comma. The following example creates a tuple of a single element.

Tup1 = (100,)

In the above case, if we do not put a comma after 100, then Python would treat **Tup1** as an integer rather than a tuple variable.

Example:

```
1.  Tup1 = (100,)
2.  type(Tup1)
3.  Tup2=(100)
4.  type(Tup2)
```

Output:

```
<class 'tuple'>
<class 'int'>
```

The `tuple()` Function

Python has a built-in function called **`tuple()`** that may be used to make tuples. While we can build tuples without utilising this function, it offers a different method. The **`tuple()`** function is used to construct a tuple in the following example.

Example:

```
1. # example of tuple() function
2. # creating an empty tuple
3. T1 = tuple()
4. print (T1)
5. # creating a tuple from List
6. T2 = tuple([1,2,3,4,5])
7. print (T2)
8. # creating a tuple from strings
9. T3 = tuple("Python Programming")
10. print (T3)
```

Output:

```
()
(1, 2, 3, 4, 5)
('P', 'y', 't', 'h', 'o', 'n', ' ', 'P', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g')
```

Accessing Tuple

To access the elements of a tuple, indexes are used. Tuple indexes start at 0, like string indices, and can be sliced, concatenated, and so on. The following examples show how we can access values in tuples.

Example:

```
1. # tuple of characters
2. tup1 = ('A', 'B', 'C', 'D')
3. # tuple of integers
4. tup2 = (10, 20, 30, 40, 50, 60, 70 )
5. # displaying first element of tuple1
6. print tup1[0]
7. # displaying second to fifth elements of tuple 2
8. print tup2[1:5]
```

Output:

```
A  
(20, 30, 40, 50)
```

Tuple Indexes

Indexes in Python can be positive or negative, like list and string indexes. Negative indices begin at -1, whereas positive indices begin at 0. While negative indices are used to access items from the tuple's end, positive indexes are used to access elements from the tuple's beginning. The below example demonstrates the use of a negative index.

Example:

```
1. Tuple1 = (10, 20, "Books", 108.92)  
2. # Accessing last element using negative index  
3. print (Tuple1[-1])  
4. # Accessing third element from last  
5. print (Tuple1[-3])
```

Output:

```
108.92  
20
```

Tuple Slicing

The action of slicing is analogous to that of strings. We need to specify the beginning and end of the slicing. The result is a tuple that lies between those two places and incorporates each of the components. We have to use the [] operator on the tuple to slice a tuple. When indexing a tuple, the index is taken from the count of the tuples on the left if the integer we are working with is positive. If the index is negative, the index is derived from the tuple count on the right. The code that follows is an illustration of slicing operations that may be performed on the tuple.

Example:

```

1. T1 = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
2. # print elements from 3rd to 6th
3. print (T1[2:6])
4. # print elements from start to 4th position
5. print (T1[:4])
6. # print elements from 4th to last
7. print (T1[3:])
8. # print elements from 6th to second last
9. print (T1[5:-1])
10. # print entire tuple
11. print (T1[:])

```

Output:

```

(3, 4, 5, 6)
(1, 2, 3, 4)
(4, 5, 6, 7, 8, 9, 10)
(6, 7, 8, 9)
(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

```

Updating Tuple

Tuples are immutable, which means that the values of the items included within a tuple will not be altered or changed in any way. However, if we want to update the nested items they are changeable. In the following piece of code, for instance, we modify the list element that is included within the tuple. Because the list's components are adjustable, it is acceptable to do so.

Example:

```

1. T1 = (1, [2, 3, 4, 5], "Python")
2. print (T1)
3. # Updating is not valid for tuples
4. #T1[0] = 100 # It generates error
5. # Changing the list element, this is valid because List is mutable
6. T1[1][2] = 10
7. print (T1)

```

Output:

```

(1, [2, 3, 4, 5], 'Python')
(1, [2, 3, 10, 5], 'Python')

```

We can create a new tuple using the portion of an existing tuple. The below code demonstrates the creation of a new tuple.

Example:

```
1. T1 = (1, 2, 3, 4.5)
2. T2 = ('ABCD', 'XYZW')
3. # New tuple creation from existing
4. T3 = T1 + T2[0:1]
5. print T3
```

Output:

```
(1, 2, 3, 4.5, 'ABCD')
```

Deleting Elements from a Tuple

We have already shown that the elements of a tuple are immutable, which means they cannot be altered or removed. However, it is best to remove the entire tuple if feasible. To remove a whole tuple, use the ***del*** statement.

Example:

```
1. T2 = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
2. # Printing tuple
3. print (T2)
4. #del T2[2]  #it will generate an error
5. # deleting an entire tuple
6. del T2
7. #print after deleting a tuple
8. print (T2)      #it will generate error
```

Output:

```
(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

Traceback (most recent call last):

File "main.py", line 11, in <module>

```
print(T2)      #it generates error
```

NameError: name 'T2' is not defined

Tuple Operations

The many operations that may be performed on tuples are relatively similar to those that can be performed on lists. In most cases involving tuples, we only encountered the + operator. It works with tuples in exactly the same way as it does with lists. Tuples respond to the + and * operators similarly to strings; they also imply concatenation and repetition in this context; however, a new tuple is a result rather than a string. When subjected to any general sequence operations, tuples behave similarly to strings. Addition operators combine all the elements of two or more tuples into a new tuple. If a tuple is multiplied by any integer, x would create another tuple with all the elements repeated x times from the first tuple. $T*5$ indicates that the elements of tuple T will be replicated five times.

Example:

```
1. T1 = (1, 2, 3)
2. T2=(4, 5)
3. T3=(10,)
4. #Addition
5. T4=T1+T2+T3
6. print(T4)
7. #Multiplication
8. print(T1*5)
9. #replicate string 3 times
10. print("Python"*3)
```

Output:

```
(1, 2, 3, 4, 5, 10)
(1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3)
PythonPythonPython
```

Tuple Basic Functions

Table [1](#) lists some fundamental functions and keywords that are applicable to tuples.

Table 1 **Tuple basic functions.**

len() This function is used to get the number of elements in the tuple.

max()

and These functions are used to get the maximum and minimum values from the tuple.

min()

sum() This function is used to get the sum of all the elements of the tuple.

This keyword is used with tuples same as strings and lists. It is used to check whether any element is present or not in the tuple. If the element is present, it returns True. Otherwise, it returns False.

In order to compare two tuples, the **cmp()** function is called upon. It will return either 1, 0, or -1 depending on whether or not the two tuples being compared are comparable. The comparison of these two tuples is handled by this function, which accepts them as arguments. If T1 and T2 are two different tuples, then the comparison will provide the following result based on what it finds.

if $T1 > T2$, then $cmp(T1, T2)$ returns 1,

if $T1 = T2$, then $cmp(T1, T2)$ returns 0

if $T1 < T2$, then $cmp(T1, T2)$ returns -1

cmp() function is only in python 2.x. It does not work in python 3.x.

It is a built-in Python function that searches for a given element in a tuple and returns the lowest index where the element appears. *index()* function can take a maximum of three arguments and follows the given syntax:

index() **tuple.index(element, start, end)**

Here, the **element** is the first argument that must be searched in the tuple. **Start** and **end** are optional arguments that indicate starting and ending point index of the searching element.

count() The *count()* function returns the number of occurrences in the given tuple for an element.

Example:

```

1.  #tuple with six elements
2.  T = (10, -2, 300, 40, 302, 900)
3.  # length of the tuple
4.  print ("Length:", len(T))
5.  # Max value from tuple
6.  print ("Max:", max(T))
7.  # Min value from tuple
8.  print ("Min:", min(T))
9.  # Sum of all elements of tuple
10. print ("Sum:", sum(T))
11. # Check whether 40 is present or not in tuple
12. print (40 in T)
13. # Check whether 500 is present or not in tuple
14. print (500 in T)
15.
16. =====index() Example
17. # Tuple of four elements
18. T1 = (100, 20, 30, 40)
19. # tuple of characters
20. T2 = ("PYTHON")
21. # print the index of 100 in tuple T1
22. print ("Index No.:", T1.index(100))
23. # print the index of T in tuple T2
24. print ("Index No.:", T2.index("T"))
25. # search the index of 20, start from 1 to 4
26. print ("Index No.:", T1.index(20, 1, 4))
27. =====count() Example
28. #tuple 1
29. T1 = (1, 1, 1, 2, 2, 2, 1, 2, 3, 1, 1)
30. #tuple 2
31. T2 = ('a', 'a', 'a', 'b', 'b', 'a', 'c', 'b', 'a', 'b')
32. #tuple 3
33. T3 = ['Bat', 'Cat', 'Sat', 'Bat', 'cat', 'Bat']
34. # Counts the number of times 1 appears in tuple 1
35. print ("Count of 1:", T1.count(1))
36. # Counts the number of times 'a' appears in tuple 2
37. print ("Count of a:", T2.count('a'))
38. # Counts the number of times 'Bat' appears in tuple 3
39. print ("Count of Bat:", T3.count('Bat'))

```

Output:

```

Length: 6
Max: 900
Min: -2
Sum: 1550
True
False
Index No.: 0
Index No.: 2
Index No.: 1
Count of 1: 6
Count of a: 5
Count of Bat: 3

```

Tuple Assignment

Tuple assignment is a very helpful function that Python provides. It is named so because it allows values to be assigned to a tuple of variables on the left-hand side of an assignment from a tuple on the right-hand side of the assignment. **Unpacking** a tuple of values into a variable is another name for this operation. When we **pack**, the data is placed inside a new tuple, but when we unpack, we remove those values and place them inside a single variable. With the assistance of this feature, it is possible to assign more than one variable at once while a sequence is displayed on the left. The following illustration shows how one tuple, T1, may be used to assign values to five distinct variables. In this task, the number of variables listed on the left-hand side of the equation must match the number of tuple elements listed on the right-hand side of the equation. If this requirement is not met, an error will be generated.

Example:

```

1. # tuple with five elements/Packing
2. T1= ('David', 'India', 1990, 1001)
3. print ("Tuple Elements are:", T1)
4. # tuple assignment to different variables/Unpacking
5. (EmpName, Country, BirthYear, EmpId)=T1
6. # printing individual variables
7. print ("-----")
8. print ("EmpName:", EmpName)
9. print ("Country:", Country)
10. print ("BirthYear:", BirthYear)
11. print ("EmpId:", EmpId)

```

Output:

```
Tuple Elements are: ('David', 'India', 1990, 1001)

-----

EmpName: David
Country: India
BirthYear: 1990
EmpId: 1001
```

In the above example, we have a tuple T1 which contains five elements and assign different values to the variables EmpName, Country, BirthYear, and EmpId in a single statement. We can also assign individual element values to variables such as:

Example:

```
1. # tuple with five elements
2. T1= ('David', 'India', 1990, 1001)
3. print ("Tuple Elements are:", T1)
4. # assigning one value at a time
5. EmpName=T1[0]
6. Country=T1[1]
7. BirthYear=T1[2]
8. EmpId=T1[3]
9. #printing individual values
10. print ('-----')
11. print (EmpName)
12. print (Country)
13. print (BirthYear)
14. print (EmpId)
```

Output:

```
Tuple Elements are: ('David', 'India', 1990, 1001)

-----

David
India
1990
1001
```

Tuples and Functions

When calling a function with the prefix `*`, a unique method can be used to get the parameters in the form of a tuple. This is useful information when the number of arguments in the function is taken as a function. Because of the `*` prefix placed on the args variable; any further arguments sent to the function will be saved in that variable as a tuple.

Example:

```

1.  #This function receives a tuple as an argument
2.  def fun1(*args):
3.      print(type(args)) #printing the type of args
4.      print(args)
5.      #creating a tuple
6.      T1=("Python", "Programming book", 2022)
7.      #passing tuple to a function
8.      fun1(T1)

```

Output:

```

<class 'tuple'>
(('Python', 'Programming book', 2022),)

```

It is not feasible to return numerous values from a function in C or other programming languages, but in Python, doing so is not only conceivable but also rather straightforward. We can return numerous values in Python by using simple returning statements that are delimited by commas. The following code snippet demonstrates how to define a function called ***fun_ret_mul()*** that will return both a string and a number.

Example:

```

1. #Returning multiple values as tuple
2. # This function returns a tuple
3. def fun_ret_mul():
4.     s1 = "Python book"
5.     x = 2022
6.     return (s1, x)
7. #calling function
8. T1= fun_ret_mul() #T1 is a tuple
9. print(type(T1))
10. print(T1)

```

Output:

```
<class 'tuple'>
('Python book', 2022)
```

Nesting of Tuples

Python allows us to declare a tuple inside another tuple; this type of tuple is referred to as a nested tuple. Nesting tuples are used whenever there is a need to organize data into hierarchical structures. If only one element is included within a tuple, then we cannot refer to it as a tuple. It should consist of a comma after it as a trailing mark to indicate to the interpreter that it is a tuple. In the following example, we have declared a tuple contained within another tuple.

Example:

```
1.  # Nested tuples
2.  Tuple1 = ("Python", [1,2,3], (5, 6.6, "Programming", "Book"))
3.  # Print all elements of tuple
4.  print("Tuple values are:", Tuple1)
5.  # Print first element
6.  print("first Element:", Tuple1[0])
7.  #print second element or sub-tuple
8.  print("sub tuple:", Tuple1[1])
9.  #print third element or sub-tuple
10. print("sub tuple:", Tuple1[2])
```

Output:

```
Tuple values are: ('Python', [1, 2, 3], (5, 6.6, 'Programming', 'Book'))
first Element: Python
sub tuple: [1, 2, 3]
sub tuple: (5, 6.6, 'Programming', 'Book')
```

Tuples from String and List

In Python, a tuple may be constructed not just from a text but also from a list. This kind of conversion is required whenever multiple types of data are available and the need to create a new type. You can convert a string or a list into a tuple by using the ***tuple()*** function. The following illustration provides further clarification of this idea.

Example:

```

1.  # List and string creation
2.  L1 = ['Python',1,2,3,4,5,6]
3.  S1 = "Programming Language"
4.  # printing original list and string
5.  print ("The original List : " , L1)
6.  print ("The original String : " , S1)
7.  print (type(L1))
8.  print(type(S1))
9.  #List to tuple creation
10. result1 = tuple(L1)
11. #String to tuple creation
12. result2=tuple(S1)
13. print ("Converted List : " , result1)
14. print (type(result1))
15. print ("Converted String : " , result2)
16. print (type(result2))

```

Output:

```

The original List : ['Python', 1, 2, 3, 4, 5, 6]
The original String : Programming Language
<class 'list'>
<class 'str'>
Converted List : ('Python', 1, 2, 3, 4, 5, 6)
<class 'tuple'>
Converted string: ('P', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g', ' ', 'L', 'a', 'n', 'g', 'u', 'a', 'g', 'e')
<class 'tuple'>

```

zip() Function

The Python zip () function is mainly used to combine data of two container elements. This function takes iterables and aggregates their elements based on their order in the iterables. As an input, it takes any number of iterables and returns an iterator of tuples. Iterables may be lists, strings, *etc.* Below is an example of group items of two different lists based on the same index.

Example:

```

1. #creating two lists
2. List1 = ['A', 'B', 'C', 'D']
3. List2 = [10, 20, 30, 40]
4. #combine using zip
5. Result = tuple(zip(List1, List2))
6. print (Result)
7. print (type(Result))

```

Output:

```
((A', 10), (B', 20), (C', 30), (D', 40))  
<class 'tuple'>
```

The returned iterator takes the length of the shortest iterator passed to the function if the passed iterables are of different lengths. In the below example, we have two lists of size four and two, respectively. The result tuple has length two, based on the shortest list **list2**.

Example:

```
1. #creating two lists  
2. List1 = ['A', 'B', 'C', 'D']  
3. List2 = [100, 200]  
4. #combine using zip  
5. Result = tuple(zip(List1, List2))  
6. print (Result)  
7. print (type(Result))
```

Output:

```
((A', 100), (B', 200))  
<class 'tuple'>
```

The `zip()` function also works for more than two iterables, such as:

Example:

```
1. #creating three tuples  
2. T1 = ('Amit', 'Sumit', 'Joy', 'Dev')  
3. T2 = (50000, 20000, 25000, 80000)  
4. T3=('HR', 'Clerk', 'IT', 'Manager')  
5. #combine using zip  
6. Result = tuple(zip(T1, T2, T3))  
7. print (Result)  
8. print (type(Result))
```

Output:

```
((Amit', 50000, 'HR'), (Sumit', 20000, 'Clerk'), (Joy', 25000, 'IT'), (Dev', 80000, 'Manager'))  
<class 'tuple'>
```

Inverse `zip(*)` Function

The `*` operator can be used with the `zip ()` function. It unzips the tuples into independent lists. In other words, the `*` operator with the `zip ()` function unpacks a sequence into positional arguments. The below example shows the concept of unzipping tuples.

Example:

```
1. # using zip() and * operator to unpack sequences
2. # creating tuple
3. T1 = [('A', 100), ('B', 200), ('C', 300), ('D', 4000)]
4. # Original tuple
5. print ("Original tuple is : ", T1)
6. # using zip() and * operator to perform unzipping
7. T2,T3 = zip(*T1)
8. T4=zip(*T1)
9. # Printing modified tuple 1
10. print ("Modified Tuple1 : ", T2)
11. print (type(T2))
12. # Printing modified tuple 2
13. print ("Modified Tuple2 : ", T3)
14. print (type(T3))
15. # Printing modified tuple 3
16. print ("Modified Tuple2 : ", tuple(T4))
```

Output:

```
Original tuple is : [('A', 100), ('B', 200), ('C', 300), ('D', 4000)]
Modified Tuple1 : ('A', 'B', 'C', 'D')
<class 'tuple'>
Modified Tuple2 : (100, 200, 300, 4000)
<class 'tuple'>
Modified Tuple2 : (('A', 'B', 'C', 'D'), (100, 200, 300, 4000))
```

Tuple Sorting

We have an in-built function named `sorted()` in Python to sort the tuple elements. This function takes one argument as a parameter to sort tuples.

```
sorted (arg1)
```

The tuple is by default sorted in ascending order using the `sorted()` function. If we want to sort in descending order, we must set the parameter `reverse` to `True`.

```
sorted (arg1, reverse=True)
```

Example:

```
1. # Create a tuple
2. Tuple1 = (20, 4, 6, -1, 4, 7.8, 2.79, 5.90)
3. # Sorting in ascending order
4. TDefault=sorted(Tuple1)
5. # Sorting in descending order
6. TReverse=sorted(Tuple1, reverse=True)
7. # Printing original tuple
8. print ("Original tuple:", Tuple1)
9. # Printing sorted tuple- Ascending
10. print ("Sorted tuple:", TDefault)
11. # Printing sorted tuple- Descending
12. print ("Sorted tuple in reverse order:", TReverse)
```

Output:

```
Original tuple: (20, 4, 6, -1, 4, 7.8, 2.79, 5.9)
```

```
Sorted tuple: [-1, 2.79, 4, 4, 5.9, 6, 7.8, 20]
```

```
Sorted tuple in reverse order: [20, 7.8, 6, 5.9, 4, 4, 2.79, -1]
```

DICTIONARY

Another essential built-in data type in Python is referred to as the dictionary. Data can be saved there in the form of collections. dictionary usage is not as widespread as other data structures, such as arrays and lists. In Python, a dictionary is a collection that stores the data or value that is associated with a certain set of keys. The relationship between values and keys is known as a mapping relation. Hence dictionary entries in Python take into account mapping relations between items. Compared to lists and arrays, the operation of dictionaries is rather distinct. The collection items are saved in the list and may be retrieved using index positions. When we wish to retrieve a particular element in a list or array, it implies that we must do a lookup using that element's index number. Programmers are not interested in working with indexing structures, yet many applications call for a more versatile method in terms of how information is retrieved from the collection. As a result, dictionaries can offer a superior option. Some concrete examples might help illustrate the meaning behind the notion of dictionaries. If we want to find out the employee's phone number based on their name, or if we want to know the city's name based on the supplied pin code, we may use this. The result is an effective data structure referred to as a dictionary and followed mapping

between different values and keys. This particular combination is referred to as a *key-value* pair in the realm of computer programming. With this combination, we can retrieve any information or value (the name of a city) that is connected with a certain key (pin code). The basic structure of the dictionary, together with an example of a *key-value* pair, is depicted in Fig (1).

A dictionary is a hash or associative array in other computer languages. This illustrative example demonstrates the use of a dictionary to store city names with their respective zip codes.

```
{ "Name": "A", "Course": "Python Programming", "Duration": 1, "Grade": 8 }
```

```
dict= {'Name': 'A', 'Course': 'Python Programming', 'Duration': 1, 'Grade': 8}
```

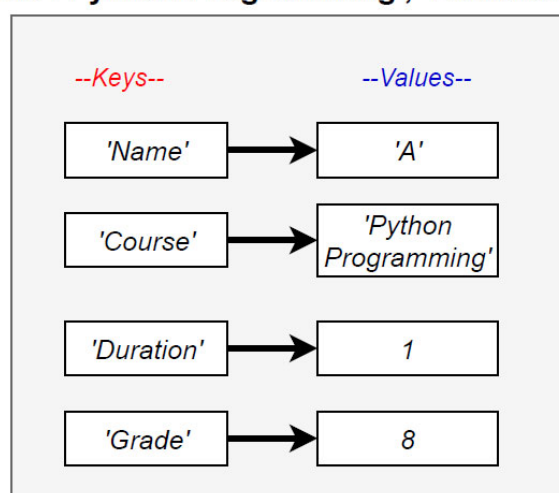


Fig. (1))

Simple structure of a dictionary.

In a Python dictionary, all the elements are enclosed in curly brackets such as { and }. A colon (:) is used to join the *key-value* pair. The following is the syntax of the dictionary.

Syntax:

```
dictionary = {
    <key1>: <value1>,
    <key2>: <value2>,
    .
    .
    .
    <keyn>: <valuen>
}
```

Dictionary Creation

Python allows the creation of dictionaries by enclosing a sequence of elements in curly braces { } and separated by commas (,). The dictionary has several values, each corresponding to a certain key. Values can be of any data type and replicated in a dictionary, whereas keys cannot be copied and are immutable. Because of this, it's important to use the correct case when entering a dictionary key. A dictionary can be made in several different ways.

The first method of dictionary creation is to create an empty dictionary and add items.

Example:

```
1. # Creating an empty dictionary
2. Dict1 = {}
3. # Adding items to Dictionary
4. Dict1["key1"] = [1, 2, 3, 4, 5]
5. Dict1["key2"] = ["India", "US", "Australia", "Japan", "China"]
6. print(Dict1)
7. # Checking type of Dict1
8. print(type(Dict1))
```

Output:

```
{'key1': [1, 2, 3, 4, 5], 'key2': ['India', 'US', 'Australia', 'Japan', 'China']}
<class 'dict'>
```

Another way of creating a dictionary is by passing key-value pairs of literals.

Example:

```
1. # Creating a dictionary
2. Dict1 = {
3.     1 : "India",
4.     2 : "US",
5.     3 : "Australia",
6.     4 : "Japan",
7. }
8. # Printing dictionary
9. print(Dict1)
10. # Checking type of Dict1
11. print(type(Dict1))
```

Output:

```
{1: 'India', 2: 'US', 3: 'Australia', 4: 'Japan'}
<class 'dict'>
```

We can create a dictionary by passing key-value pairs to `dict()` function.

Example:

```
1. # Creating a dictionary
2. Dict2 = dict(
3.     Name='David',
4.     Age=55,
5.     Address='Canada',
6.     Salary='$5000')
7. # Printing dictionary
8. print (Dict2)
9. # Checking type of Dict1
10. print (type(Dict2))
```

Output:

```
{'Name': 'David', 'Age': 55, 'Address': 'Canada', 'Salary': '$5000'}
<class 'dict'>
```

The next method of creating dictionaries is to pass the list of tuples to `dict ()` function.

Example:

```
1. # Passing list of tuples
2. Dict1= dict(
3.     [('Name','William'),
4.     ('Age',40),
5.     ('Address','Japan'),
6.     ('Salary','$5000')])
7. # Printing dictionary
8. print (Dict1)
9. # Checking type of Dict1
10. print (type(Dict1))
```

Output:

```
{'Name': 'William', 'Age': 40, 'Address': 'Japan', 'Salary': '$5000'}
<class 'dict'>
```

Accessing Dictionary Values

Accessing any value in a dictionary is accomplished by using *key* names but accessing values in other forms of data is accomplished by indexing. To retrieve the values associated with a key, you can use the name of the key enclosed in square brackets []. The detailed syntax below may be utilized to obtain any value connected with the key in the dictionary.

Syntax:

```
Dictionaryname[keyname]
```

Example:

```
1.  # Creating a dictionary
2.  D1= {
3.      101 : "James",
4.      102 : "William",
5.      103 : "John",
6.      104 : "Sam",
7.  }
8.  # Retrieving values associated with keys
9.  print ('101:',D1[101])
10. print ('103:',D1[103])
11. print ('104:',D1[104])
```

Output:

```
101: James
103: John
104: Sam
```

Use the **get()** function with the key to acquire the corresponding value from the dictionary. This is an alternative method. The following are some possible applications of this function:

Example:

```
1.  # Creating a dictionary
2.  D1= {
3.      101 : "James",
4.      102 : "William",
5.      103 : "John",
6.      104 : "Sam",
7.  }
8.  # Retrieving values associated with keys
9.  print ('101:',D1.get(101))
10. print ('103:',D1.get(103))
11. print ('104:',D1.get(104))
```

Output:

```
101: James
103: John
104: Sam
```

Modifying Dictionary

As a mutable data type, dictionaries allow the element values to be altered whenever necessary. By combining an assignment operator with keys, we can either add new items or change the value of existing ones. If the key is already in use, then the value will be modified to reflect the addition of the new key. If the key does not already exist in the dictionary, it will be replaced with a new *key-value* pair. When adding a new entry to a dictionary, the syntax that is shown above should be used.

Syntax:

```
Dictionaryname[key]=value
```

Example:

```
1.  # Creating a dictionary
2.  D1= {
3.      101 : "James",
4.      'age' : 40,
5.      'city' : "Delhi",
6.      }
7.  # Print the elements
8.  print (D1)
9.  # Add a new element
10. D1['Post'] = 'Manager'
11. # After adding new element, print all values
12. print (D1)
13. # modifying and existing value
14. D1['age'] = 50
15. # After modifying value, print all values
16. print (D1)
```

Output:

```
{101: 'James', 'age': 40, 'city': 'Delhi'}
{101: 'James', 'age': 40, 'city': 'Delhi', 'Post': 'Manager'}
{101: 'James', 'age': 50, 'city': 'Delhi', 'Post': 'Manager'}
```

As we have seen in the above example, if a key (age) exists in a dictionary, then it modifies or replaces the old value (40) with the new value (50).

Delete an Element from the Dictionary

You can remove any element from the dictionary by utilizing the corresponding keys for that element. To remove a component from the dictionary that is currently being used, you can make use of the “del” operator. The disadvantage of utilizing this is that an exception will be thrown if the key is not there. This is something that should be considered before using it. The syntax that is presented below is what is utilized when removing an element from a dictionary.

Syntax:

```
del Dictionaryname[key]
```

The below example deletes an element such as age with the associated value from the dictionary D1.

Example:

```
1.  # Creating a dictionary
2.  D1= {
3.      101 : "James",
4.      'age' : 40,
5.      'city' : "Delhi",
6.      }
7.  # Print the elements
8.  print (D1)
9.  # Add a new element
10. D1['Post'] = 'Manager'
11. print ("Available Elements are:", D1)
12. # After adding new element, print all values
13. print ("Before Deletion:",D1)
14. # deleting an element age
15. del D1['age']
16. # Values after deleted an element
17. print ("After Deletion:", D1)
```

Output:

```
{101: 'James', 'age': 40, 'city': 'Delhi'}
Available Elements are: {101: 'James', 'age': 40, 'city': 'Delhi', 'Post': 'Manager'}
Before Deletion: {101: 'James', 'age': 40, 'city': 'Delhi', 'Post': 'Manager'}
After Deletion: {101: 'James', 'city': 'Delhi', 'Post': 'Manager'}
```

The del operator can even completely delete the dictionary, such as:

```
del D1
```

After deleting dictionary **D1**, if we try to access **D1**, it will cause an error because **D1** no longer exists. The *pop()* function is another approach that may be used to remove a key and the associated value from a dictionary. The benefit of using this instead of the *del* operator is that if you try to delete a non-existent element, it will output the value or message you want rather than fail. Second, in addition to finishing a fundamental delete operation, it also returns the value of a key that is being removed. This is in addition to the fact that it deletes the key.

Example:

```

1.  # Initializing a dictionary
2.  D1= {"John" : 30, "Sam" : 25, "James" : 28, "David" : 29, "Bob" :30}
3.  # Printing dictionary before element deletion
4.  print ("The dictionary before performing deletion is : " , D1)
5.  # Using pop() function to delete an element such as John
6.  del_value = D1.pop('John')
7.  # Printing dictionary after element deletion
8.  print ("The dictionary after deletion is : " , D1 )
9.  print ("The removed key's value is : " , del_value)
10. # Again deleting an element which is not present
11. # assigns 'Not available' to del_value
12. del_value1 = D1.pop('Jai', 'Not available')
13. # Printing dictionary after element deletion
14. print ("The Dictionary after remove is : " , D1)
15. print ("The removed key's value is : " , del_value1)

```

Output:

```

The dictionary before performing deletion is : {'John': 30, 'Sam': 25, 'James': 28, 'David': 29, 'Bob': 30}

The dictionary after deletion is : {'Sam': 25, 'James': 28, 'David': 29, 'Bob': 30}

The removed key's value is : 30

The dictionary after remove is : {'Sam': 25, 'James': 28, 'David': 29, 'Bob': 30}

The removed key's value is : Not available

```

The Clear () Function

By utilizing the *clear ()* function, we can remove all of the dictionary's entries simultaneously. The following example removes all items from the dictionary known as D1.

Example:

```

1.  # Example of deleting all element using clear () function
2.  # Initializing a dictionary
3.  D1= {"Amit" : 20, "Sumit" : 25, "John" : 28, "David" : 29, "Bob" :30}
4.  # Printing dictionary before element deletion
5.  print ("The dictionary before performing deletion is : " , D1)
6.  # Deleting all elements
7.  D1.clear()
8.  # Printing dictionary after element deletion
9.  print ("The Dictionary after remove is : " , D1)

```

Output:

The dictionary before performing deletion is : {'Amit': 20, 'Sumit': 25, 'John': 28, 'David': 29, 'Bob': 30}

The dictionary after remove is : {}

Sorting Dictionaries

An element in a dictionary is made up of a key and the value that corresponds to it. The process of sorting entries in a dictionary may be carried out as a parameter using either the key or the value components. dictionary components can be sorted either by their keys or their values with the help of the *sorted()* method.

Example:

```
1. # Initializing a dictionary
2. D1= {"James" : 50, "William" : 30, "John" : 25, "David" : 60, "Bob" :20}
3. print (type(D1))
4. # Printing dictionary before sorting
5. print ("The original dictionary is: ", D1)
6. # Sorting by key
7. D2= dict(sorted(D1.items(), key=lambda x: x[0]))
8. print ("After sorting by key: ", D2)
9. print (type(D2))
```

Output:

```
<class 'dict'>
The original dictionary is: {'James': 50, 'William': 30, 'John': 25, 'David': 60, 'Bob': 20}
After sorting by key: {'Bob': 20, 'David': 60, 'James': 50, 'John': 25, 'William': 30}
<class 'dict'>
```

In this case, the *D1.items()* method produces a list of tuples that each include the keys and their corresponding values. The lambda function will return the key for a certain item tuple when it is called (0th element of the dictionary). After these have been given to the *sorted()* method, the latter will produce a sorted sequence, which will then be entered into a dictionary.

This method may be used in Python versions 3.6 and later since it similarly considers ordered sequences and dictionaries. In previous versions, we may replace the lambda function located in the operator module with the *itemgetter()* described in the following example.

Example:

```
1. from operator import itemgetter
2. # Initializing a dictionary
3. D1= {"James" : 50, "William" : 30, "John" : 25, "David" : 60, "Bob" :20}
4. print (type(D1))
5. # Printing dictionary before sorting
6. print ("The original dictionary is: ", D1)
7. # Sorting by key
8. D2= dict(sorted(D1.items(), key=itemgetter(0)))
9. print ("After sorting by key: ", D2)
10. print (type(D2))
```

Output:

```
<class 'dict'>
The original dictionary is: {'James': 50, 'William': 30, 'John': 25, 'David': 60, 'Bob': 20}
After sorting by key: {'Bob': 20, 'David': 60, 'James': 50, 'John': 25, 'William': 30}
<class 'dict'>
```

Sorting Dictionary by Value

It is a good analogy to ordering a dictionary by its keys rather than its values. The sole difference is that the value component of this kind's corresponding element will serve as the parameter on which the sorting will be carried out. The dictionary is arranged in descending order by the value argument in the following example:

Example:

```
1. # Initializing a dictionary
2. D1= {"James" : 50, "William" : 30, "John" : 25, "David" : 60, "Sam" :20}
3. print (type(D1))
4. # Printing dictionary before sorting
5. print ("The original dictionary is: ", D1)
6. # Sorting by value
7. D2= dict(sorted(D1.items(), key=lambda x: x[1]))
8. print ("After sorting by key: ", D2)
9. print (type(D2))
```

Output:

```
<class 'dict'>
The original dictionary is: {'James': 50, 'William': 30, 'John': 25, 'David': 60, 'Sam': 20}
After sorting by key: {'Sam': 20, 'John': 25, 'William': 30, 'James': 50, 'David': 60}
<class 'dict'>
```

In the above example, the dictionary D1 is sorted according to the values returned by the lambda function (x [1] value for x element). We use the following code for the older versions of Python.

Example:

```

1. from operator import itemgetter
2. # Initializing a dictionary
3. D1= {"James" : 50, "William" : 30, "John" : 25, "David" : 60, "Sam" :20}
4. print (type(D1))
5. # Printing dictionary before sorting
6. print ("The original dictionary is: ", D1)
7. # Sorting by value
8. D2= dict(sorted(D1.items(), key=itemgetter(1)))
9. print ("After sorting by key: ", D2)
10. print (type(D2))

```

Output:

```

<class 'dict'>
The original dictionary is: {'James': 50, 'William': 30, 'John': 25, 'David': 60, 'Sam': 20}
After sorting by key: {'Sam': 20, 'John': 25, 'William': 30, 'James': 50, 'David': 60}
<class 'dict'>

```

Sorting Dictionary in Reverse Order

Another input called reverse is taken into consideration by the *sorted()* method. The sequence in which the sorting should be carried out may be determined using this information, which can be used as a guide. If *True* is passed, the sorting will be done in the other direction, which is called descending order. In addition, if the default value of *False* is used, the sorting will be carried out ascendingly.

Example:

```

1. # Initializing a dictionary
2. D1= {"James" : 50, "William" : 30, "John" : 25, "David" : 60, "Sam" :20}
3. # Printing dictionary before sorting
4. print ("The original dictionary is: ", D1)
5. # Sorting by key in reverse order
6. D2= dict(sorted(D1.items(),reverse=True, key=lambda x: x[0]))
7. print ("After Reverse order Sorting by key: ", D2)
8. # Sorting by value in reverse order
9. D3= dict(sorted(D1.items(),reverse=True, key=lambda x: x[1]))
10. print ("After Reverse order Sorting by value: ", D3)

```

Output:

```
The original dictionary is: {'James': 50, 'William': 30, 'John': 25, 'David': 60, 'Sam': 20}
After Reverse order Sorting by key: {'William': 30, 'Sam': 20, 'John': 25, 'James': 50, 'David': 60}
After Reverse order Sorting by value: {'David': 60, 'James': 50, 'William': 30, 'John': 25, 'Sam': 20}
```

Iterating Over a Dictionary

Although the dictionary itself is not an iterable object, the methods `items()`, `keys()`, and `values` produce iterable view objects that may be used for iterating through the dictionary's entries. We can use a for loop to go through all the data's key-value pairs. Using the `items()` function will return a list of tuples, with each tuple consisting of a key and value pair.

Example:

```
1. # Creating a dictionary
2. Dict1= {0 : 'Zero', 1 : 'One', 2 : 'Two', 3 : 'Three', 4 : 'Four', 5 : 'Five'}
3. # Printing dictionary
4. print ("The original dictionary is: ", Dict1)
5. # Iterating with for loop
6. print ('Iterating with for loop:')
7. for t in Dict1.items():
8.     print (t)
```

Output:

```
The original dictionary is: {0: 'Zero', 1: 'One', 2: 'Two', 3: 'Three', 4: 'Four', 5: 'Five'}
Iterating with for loop:
(0, 'Zero')
(1, 'One')
(2, 'Two')
(3, 'Three')
(4, 'Four')
(5, 'Five')
```

We can also iterate the dictionary by storing the key and value out of each pair in two separate variables such as.

Example:

```
1. # Creating a dictionary
2. Dict1= {0 : 'Zero', 1 : 'One', 2 : 'Two', 3 : 'Three', 4 : 'Four', 5 : 'Five'}
3. # Printing original dictionary
4. print ("The original dictionary is: " , Dict1)
5. # Iterating with for loop and storing key and value separately
6. print ('Iterating with storing separate values:')
7. for key,value in Dict1.items():
8.     print (key,value)
```

Output:

```
The original dictionary is: {0: 'Zero', 1: 'One', 2: 'Two', 3: 'Three', 4: 'Four', 5: 'Five'}
Iterating with storing separate values:
0 Zero
1 One
2 Two
3 Three
4 Four
5 Five
```

Another way of dictionary iteration is to use the `key()` function. Using `key()` function, the associated value can be obtained as follows.

Example:

```
1. # Creating a dictionary
2. Dict1= {0 : 'Zero', 1 : 'One', 2 : 'Two', 3 : 'Three', 4 : 'Four', 5 : 'Five'}
3. # Printing original dictionary
4. print ("The original dictionary is: " , Dict1)
5. # Iterating with key() function
6. print ('Iterating using key() function:')
7. for k1 in Dict1.keys():
8.     print (k1, Dict1.get(k1))    #or print(k1, Dict1[k1])
```

Output:

```
The original dictionary is: {0: 'Zero', 1: 'One', 2: 'Two', 3: 'Three', 4: 'Four', 5: 'Five'}
```

```
Iterating using key() function:
```

```
0 Zero
```

```
1 One
```

```
2 Two
```

```
3 Three
```

```
4 Four
```

```
5 Five
```

Nested Dictionaries

A dictionary is a collection of components that are not arranged in any particular sequence, as we have seen previously in this lesson. It has a variety of components in the form of *key-value* pairs that are enclosed in curly brackets. The term “nested dictionary” refers to a situation in which one dictionary is kept inside another dictionary. It works much the same way that nested records and nested structures do in other programming languages. To put it another way, we may argue that it is a compilation of several dictionaries into a single, unified dictionary.

Syntax:

```
Nested_Dict1 = {  
    'Dict1': {'key1': 'value1'},  
    'Dict2': {'key1': 'value1'},  
    'Dict3': {'key1': 'value1'}  
}
```

Example:

```

1.  # Nested dictionary
2.  Emp = {'e1': {'name': 'James', 'age': '50', 'Post': 'Manager'},
3.         'e2': {'name': 'william', 'age': '45', 'Post': 'Clerk'},
4.         'e3': {'name': 'David', 'age': '35', 'Post': 'HR'}}
5.
6.  # Printing complete nested dictionary
7.  print (Emp)
8.
9.  print ('-----Details of e1 dictionary-----')
10. print (Emp['e1']['name'])
11. print (Emp['e1']['age'])
12. print (Emp['e1']['Post'])
13.
14. print ('-----Details of e2 dictionary-----')
15. print (Emp['e2']['name'])
16. print (Emp['e2']['age'])
17. print (Emp['e2']['Post'])
18.
19. print ('-----Details of e3 dictionary-----')
20. print (Emp['e3']['name'])
21. print (Emp['e3']['age'])
22. print (Emp['e3']['Post'])

```

Output:

```

{'e1': {'name': 'James', 'age': '50', 'Post': 'Manager'}, 'e2': {'name': 'william', 'age': '45', 'Post': 'Clerk'}, 'e3': {'name': 'David', 'age': '35', 'Post': 'HR'}}

-----Details of e1 dictionary-----

James

50

Manager

-----Details of e2 dictionary-----

william

45

Clerk

-----Details of e3 dictionary-----

David

35

HR

```

The following code snippet demonstrates how to extend a nested dictionary with a new sub-dictionary. If we want to create a new dictionary, we need to access the element by its key and provide it with a corresponding value.

Example:

```

1. Emp = {'e1': {'name': 'James', 'age': '50', 'Post': 'Manager'},
2.       'e2': {'name': 'william', 'age': '45', 'Post': 'Clerk'},
3.       'e3': {'name': 'David', 'age': '35', 'Post': 'HR'}}
4.
5. # Printing dictionary elements before adding a sub dictionary
6. print('Nested Dictionary before adding a sub dictionary:', Emp)
7. # Adding a sub dictionary
8. Emp['e4'] = {'name': 'Bob', 'age': '32', 'Post': 'Manager'}
9. # Printing dictionary elements after adding a sub dictionary
10. print('Nested Dictionary after adding a sub dictionary:', Emp)

```

Output:

Nested dictionary before adding a sub dictionary: {'e1': {'name': 'James', 'age': '50', 'Post': 'Manager'}, 'e2': {'name': 'william', 'age': '45', 'Post': 'Clerk'}, 'e3': {'name': 'David', 'age': '35', 'Post': 'HR'}}

Nested dictionary after adding a sub dictionary: {'e1': {'name': 'James', 'age': '50', 'Post': 'Manager'}, 'e2': {'name': 'william', 'age': '45', 'Post': 'Clerk'}, 'e3': {'name': 'David', 'age': '35', 'Post': 'HR'}, 'e4': {'name': 'Bob', 'age': '32', 'Post': 'Manager'}}

Updating Nested Dictionary

It is simple to alter the dictionaries' nested parts. Changing the value of an object is as easy as referring to it by its key. The existing key's value is overwritten with the new one if it is found in the dictionary. The following code snippet shows how to modify the e1 dictionary values within the Emp dictionary.

Example:

```

1. # Nested dictionary
2. Emp = {'e1': {'name': 'James', 'age': '50', 'Post': 'Manager'},
3.       'e2': {'name': 'william', 'age': '45', 'Post': 'Clerk'},
4.       'e3': {'name': 'David', 'age': '35', 'Post': 'HR'}}
5.
6. # Printing dictionary elements before update
7. print('Nested Dictionary before update:', Emp)
8. # Updating values of D1
9. Emp['e1'] = {'name': 'Bob', 'age': '32', 'Post': 'Director'}
10.
11. # Printing dictionary elements after update
12. print('Nested Dictionary after updating values:', Emp)

```

Output:

Nested dictionary before update: {'e1': {'name': 'James', 'age': '50', 'Post': 'Manager'}, 'e2': {'name': 'william', 'age': '45', 'Post': 'Clerk'}, 'e3': {'name': 'David', 'age': '35', 'Post': 'HR'}}

Nested dictionary after updating values: {'e1': {'name': 'Bob', 'age': '32', 'Post': 'Director'}, 'e2': {'name': 'william', 'age': '45', 'Post': 'Clerk'}, 'e3': {'name': 'David', 'age': '35', 'Post': 'HR'}}

>

Deleting Elements from a Nested Dictionary

We can use *del* operator to delete elements from the nested dictionary. The below example deletes the value of age according to the given key (age) from the D2 dictionary.

Example:

```
1. # Creating a nested dictionary
2. Emp = {'D1' : {'name': 'Amit', 'age': '30', 'Post': 'Manager'},
3.        'D2' : {'name': 'Joy', 'age': '25', 'Post': 'Clerk'},
4.        'D3' : {'name': 'David', 'age': '35', 'Post': 'HR'}}
5.
6. # Printing dictionary elements before delete
7. print('Nested Dictionary before delete:')
8. print(Emp)
9. # Deleting element from D2
10. del Emp['D2']['age']
11. # Printing dictionary elements after Delete
12. print('Nested Dictionary after Deleting values:')
13. print(Emp['D2'])
```

Output:

Nested dictionary before delete:

```
{'D1': {'name': 'Amit', 'age': '30', 'Post': 'Manager'}, 'D2': {'name': 'Joy', 'age': '25', 'Post': 'Clerk'}, 'D3': {'name': 'David', 'age': '35', 'Post': 'HR'}}
```

Nested Dictionary after Deleting values:

```
{'name': 'Joy', 'Post': 'Clerk'}
```

Deleting a Dictionary from the Nested Dictionary

We can delete a complete sub-dictionary from the nested dictionary using *del* operator same as elements. The below example deletes the D2-sub dictionary from the nested dictionary Emp.

Example:

```

1. # Creating a nested dictionary
2. Emp = {'D1': {'name': 'Amit', 'age': '30', 'Post': 'Manager'},
3.        'D2': {'name': 'Joy', 'age': '25', 'Post': 'Clerk'},
4.        'D3': {'name': 'David', 'age': '35', 'Post': 'HR'}}
5.
6. # Printing dictionary elements before Deleting a sub dictionary
7. print ('Nested Dictionary before delete:')
8. print (Emp)
9. # Deleting D2
10. del Emp['D2']
11. # Printing dictionary after Deleting a sub dictionary
12. print ('Nested Dictionary after Deleting D2:')
13. print (Emp)

```

Output:

Nested dictionary before delete:

```
{'D1': {'name': 'Amit', 'age': '30', 'Post': 'Manager'}, 'D2': {'name': 'Joy', 'age': '25', 'Post': 'Clerk'}, 'D3': {'name': 'David', 'age': '35', 'Post': 'HR'}}
```

Nested Dictionary after Deleting D2:

```
{'D1': {'name': 'Amit', 'age': '30', 'Post': 'Manager'}, 'D3': {'name': 'David', 'age': '35', 'Post': 'HR'}}
```

Iterating Through a Nested Dictionary

We can traverse through each element in a nested dictionary using the *for* loops such as.

Example:

```

1. # Creating a nested dictionary
2. Emp = {'D1': {'name': 'Amit', 'age': '30', 'Post': 'Manager'},
3.        'D2': {'name': 'Joy', 'age': '25', 'Post': 'Clerk'},
4.        'D3': {'name': 'David', 'age': '35', 'Post': 'HR'}}
5.
6. # Iterating dictionary elements using for loop
7. for Dict_id, value in Emp.items():
8.     print ("\n Dict ID:", Dict_id)
9.     for key in value:
10.        print (key + ': ', value[key])
11.    print ("-----")

```

Output:

```

Dict ID: D1
name: Amit
age: 30
Post: Manager
-----
Dict ID: D2
name: Joy
age: 25
Post: Clerk
-----
Dict ID: D3
name: David
age: 35
Post: HR
-----

```

In the above example, the first loop returns all the keys in the nested dictionary **Emp**. It consists of the Dict Id of each employee. We have used these IDs to unpack the values of each employee. The second loop goes through the values of each employee. Then, it returns all of the keys' name, age, and post of each employee's dictionary. Then the keys of the employees and the associated values are printed.

Built-in Dictionary Functions

Python provides its users with a wide variety of built-in functions and methods that may be utilized for the purpose of carrying out a variety of actions on dictionaries. The following are some of the most significant functions:

len() This function returns the total length of the dictionary elements. This would be equal to the number of elements in the dictionary. It can be used as follows:

str() This function produces a printable string representation of a given dictionary. It has the following syntax.

type() It returns the type of the passed parameter. If the passed parameter is a dictionary, then it would return type as a dictionary.

The comparison of two dictionary entries using their respective keys and values is handled by the `cmp()`. It will return either 1, 0, or -1 depending on whether or not the two ***cmp()*** dictionaries that are being compared are identical to one another. In order to do a dictionary comparison, this function requires two dictionaries as input parameters. The following result will be produced as a direct consequence of the comparison if D1 and D2 are two different dictionaries. This functionality is not available in Python 3.x.

This function check whether any key is True or not. It takes an iterable parameter such as ***any()*** dictionary. If all keys are False or the dictionary is empty, it returns False. If at least one key is true, it returns True.

When all elements are True in the given dictionary, this function returns True. If not, False ***all()*** will return it. In other words, If all keys are True or the dictionary is empty, True is returned by `all()` function. Else, like all other cases, this returns False.

Example:

```

1.  # Built-in dictionary functions
2.  # Creating a dictionary
3.  D1= {"James" : 30, "William" : 25, "John" : 45, "David" : 40, "Bob" :25}
4.  # Finding the length of the dictionary
5.  print ("The length of dictionary is : " , len(D1))
6.  # String representation of Dictionary
7.  D2=str(D1)
8.  print ("The String representation of dictionary is : " , D2)
9.  print (type(D2))
10. print ('The type of D1 is:', type(D1))
11. #-----any()-----
12. print("-----any()-----")
13. D1 = {0: 'It is False'}
14. print (any(D1))
15. # 1 or True
16. D2 = {0: 'It is False', 1: 'It is True'}
17. print (any(D2))
18. # Both keys are false
19. D3 = {0: 'It is False', False: 'It is False'}
20. print (any(D3))
21. # Empty Dictionary
22. D4 = {}
23. print (any(D4))
24. # All are True
25. D5 = {True: 'It is True', 1: 'It is True'}
26. print (any(D5))
27. #-----all()-----
28. print("-----all()-----")
29. D1 = {0: 'It is False'}
30. print (all(D1))
31. # 1 or True
32. D2 = {0: 'It is False', 1: 'It is True'}
33. print (all(D2))
34. # Both keys are false
35. D3 = {0: 'It is False', False: 'It is False'}
36. print (all(D3))
37. # Empty Dictionary
38. D4 = {}
39. print (all(D4))
40. # 1 or True
41. D5 = {True: 'It is True', 1: 'It is True'}
42. print (all(D5))

```

Output:

```

The length of dictionary is : 5

The String representation of dictionary is : {'James': 30, 'William': 25, 'John': 45, 'David': 40,
'Bob': 25}

<class 'str'>

The type of D1 is: <class 'dict'>

-----any()-----

False
True
False
False
True

-----all()-----

False
False
False
True
True

>

```

The *Copy ()* Method

The *copy ()* method returns a shallow copy of the dictionary. The original dictionary does not change it. The following example illustrates the use of *copy ()* method.

Example:

```

1. D1 = {'name': 'Python', 'year': 2023}
2. print ('Original Value of D1:', D1)
3. # creating a copy of D1
4. D2 = D1.copy()
5. print ('Copied Version :', D2)

```

Output:

```

Original Value of D1: {'name': 'Python', 'year': 2023}

Copied Version: {'name': 'Python', 'year': 2023}

```

Formatting Dictionaries

dictionary lookups allow us to perform an operation known as “interpolation.” Because of the syntax they use, we need to enclose the key in parentheses and place it between the % operator and the character that represents the conversion. For example, if we want to format an int that is stored in a 'salary' key to look like 'xxxx.xx', we may use % (salary).2f in the spot where we want it to be presented. This will give us the desired result.

Example:

```
1. # Creating a dictionary
2. D1= {'name': 'Sam', 'salary': 30000, 'Post': 'Manager'}
3. # Printing dictionary
4. print (D1)
5. print ('-----')
6. print ('-----Formatted dictionary-----')
7. # Formatted printing
8. print ("My Name is: %(name)s" %D1)
9. print ("My Salary is: %(salary).2f and POST is: %(Post)s " %D1)
```

Output:

```
{'name': 'Sam', 'salary': 30000, 'Post': 'Manager'}
-----
-----Formatted dictionary-----
My Name is: Sam
My Salary is: 30000.00 and POST is: Manager
```

CONCLUDING REMARKS

This chapter introduced tuples and dictionaries and covered their fundamental ideas and characteristics. We also spoke about the many procedures that may be carried out on these sequences. The principles are explained logically with the help of several examples and Python code. You can learn to code by emulating these examples. Python's file-handling concepts will be discussed in the next chapter.